Inventory Management System Case Study

Name: Akansha More

Mobile No: 7058741949

Email id: akanshabhoi3@gmail.com

Part 1:- Code Review & Debugging

During the review of the API endpoint, I identified several critical issues related to validation, database operations, and adherence to business requirements. Addressing these concerns is essential to ensure data consistency, system stability, and scalability.

Task:- List all problems you see with this code (technical and business logic)

Explain Impact: For each issue, explain what could go wrong in production

Provide Fixes: Write the corrected version with explanations

1.Multiple Database Commits

Issue:
The code commits product and inventory separately. This breaks atomicity of the operation.

Impact:
If inventory creation fails, product still gets saved. This results in inconsistent and incomplete data.

How to Fix:
Use a single transaction for both operations.Commit only after both product and inventory are created successfully.

2. No Input Validation

Issue:
The API directly accesses fields like data['name'] and data['sku'] without validation.
If any required field is missing, it can cause a runtime error.

Impact:

If required fields are not provided in the request, it can result in runtime errors (KeyError) and break the API execution.

How to Fix:

Validate all required fields before processing the request. Return an error message if any field is missing.

3.Optional Field Not Handled

Issue:

The code assumes initial_quantity is always present.It may fail if this field is missing.

Impact:

API crashes when optional data is not provided. This reduces flexibility of the system.

How to Fix:

Use .get() method with a default value. Handle missing fields safely without breaking the flow.

4. No Error Handling

Issue:

There is no exception handling in the code. Any failure will crash the API.

Impact:

Users may receive unclear or no response. System becomes unreliable in production.

How to Fix:

Use try-except blocks to handle errors. Rollback the transaction and return proper error messages.

5.Price Precision Issue

Issue:

Price is stored without ensuring decimal precision. Floating point errors can occur.

Impact:

Incorrect financial calculations may happen. This can affect billing and reporting.

How to Fix:

Use Decimal type for storing price values. Ensure consistent precision handling in the database.

6. No Warehouse Validation

Issue:

The code assumes the warehouse exists. There is no verification of warehouse_id.

Impact:
Invalid references may be created. This breaks data integrity in the system.

How to Fix:
Check if warehouse exists before inserting data. Return an error if warehouse is invalid.

7. Duplicate Inventory Possibility

Issue:
No check for existing inventory entry.
Same product-warehouse pair can be duplicated.

Impact:
Inventory data becomes inconsistent.
Stock counts may become incorrect.

How to Fix:
Check if inventory already exists before inserting.
Add a unique constraint on product_id and warehouse_id.

8. No SKU Uniqueness Check

Issue:
There is no check to ensure SKU is unique.
Duplicate products can be created with the same SKU.

Impact:
It can create confusion in product identification.
This may lead to incorrect inventory tracking.

Fix:
Check if SKU already exists before inserting a new product.
Add a unique constraint at the database level as well.

Overall, these issues highlight the need for better validation, transaction management, and alignment with business requirements.

Part 2:- Database Design

Based on the given requirements, I designed a database schema that supports multiple warehouses per company, product storage across warehouses, supplier relationships, and bundle products. Since some requirements were incomplete, I also made a few assumptions during the design.

1.Company Table:-

```
CREATE TABLE Company (
id INT PRIMARY KEY,
name VARCHAR(100)
);
```

This table stores basic information about companies using the system.

2.Warehouse Table:-

```
CREATE TABLE Warehouse (
id INT PRIMARY KEY,
company_id INT,
name VARCHAR(100),
```

```
location VARCHAR(100),
FOREIGN KEY (company_id) REFERENCES Company(id)
);
```

The Warehouse table stores details of warehouses associated with companies.
It establishes a relationship where each warehouse belongs to one company using **company_id.**

3.Product Table:-

```
CREATE TABLE Product (
id INT PRIMARY KEY,
name VARCHAR(100),
sku VARCHAR(50) UNIQUE,
price DECIMAL(10,2),
product_type VARCHAR(20)

);
```

The Product table stores details of products, including name, unique SKU, and price.
It also supports different product types (e.g., normal or bundle) for flexible product management.

4.Inventory Table:-

```
CREATE TABLE Inventory (
id INT PRIMARY KEY,
product_id INT,
warehouse_id INT,
quantity INT,
FOREIGN KEY (product_id) REFERENCES Product(id),
FOREIGN KEY (warehouse_id) REFERENCES Warehouse(id)

);
```

The Inventory table tracks the quantity of each product available in different warehouses.
It creates a link between Product and Warehouse, supporting multi-warehouse inventory management.

5.Inventory Log Table:-

```
CREATE TABLE Inventory_Log (
id INT PRIMARY KEY,
product_id INT,
warehouse_id INT,
change_quantity INT,
timestamp DATETIME

);
```

The Inventory_Log table maintains a history of inventory changes for products in warehouses.It records quantity updates along with timestamps for tracking and auditing purposes.

6.Supplier Table:-

```
CREATE TABLE Supplier (
id INT PRIMARY KEY,
name VARCHAR(100),
contact_email VARCHAR(100));
```

The Supplier table stores information about suppliers who provide products to companies. It includes basic details such as name and contact email for communication purposes.

7.Product-Supplier Mapping:-

```
CREATE TABLE Product_Supplier (
product_id INT,
supplier_id INT,
PRIMARY KEY (product_id, supplier_id),
FOREIGN KEY (product_id) REFERENCES Product(id),
FOREIGN KEY (supplier_id) REFERENCES Supplier(id));
```

The Product_Supplier table defines a many-to-many relationship between products and suppliers. It enables multiple suppliers to provide a product and supports flexible sourcing.

8.Bundle Items Table:-

```
CREATE TABLE Bundle_Items (
bundle_id INT,
product_id INT,
quantity INT,
PRIMARY KEY (bundle_id, product_id),
FOREIGN KEY (bundle_id) REFERENCES Product(id),
FOREIGN KEY (product_id) REFERENCES Product(id));
```

The Bundle_Items table represents bundle products by linking them to individual products. It specifies the quantity of each product included in a bundle, supporting composite product structures.

2. Identify Gaps (Questions)

While designing the schema, I noticed some missing details like:-

1. How is low stock threshold defined for products?
2. What is considered “recent sales activity”?
3. Can a product have multiple suppliers or only one primary supplier?
4. Are bundle products fixed or can they change over time?

5. Should inventory transfers between warehouses be tracked?
6. Do we need to store supplier pricing or lead time?

3. Explain Design Decisions:-

- I used a separate Inventory table to support many-to-many relationship between products and warehouses.
- SKU is marked as UNIQUE to avoid duplicate product entries.
- I added an Inventory_Log table to track changes in stock over time.
- A Product_Supplier mapping table is used to support multiple suppliers per product.
- Bundle products are handled using a self-referencing table (Bundle_Items).
- Indexes can be added on frequently used fields like sku, product_id, and warehouse_id for better performance.

This design is flexible and scalable, and it supports real-world inventory management scenarios such as multi-warehouse storage, supplier relationships, and bundle products.

Part 3:- API Implementation (Low Stock Alerts)

I first fetch all inventory records for the given company. Then I check each product's stock against a threshold based on product type. I filter only those products that have low stock and recent sales. Finally, I include supplier details and return the alerts.

Implementation:-

```
@GetMapping("/api/companies/{companyId}/alerts/low-stock")
public Map<String, Object> getLowStockAlerts(@PathVariable int companyId) {
    List<Map<String, Object>> alerts = new ArrayList<>();
    // Step 1: Get all inventories for company warehouses
    List<Inventory> inventories = inventoryRepository.findByCompanyId(companyId);

    for (Inventory inv : inventories) {
        Product product = inv.getProduct();
        Warehouse warehouse = inv.getWarehouse();

        // Step 2: Define threshold based on product type
        int threshold;
```

```
if ("normal".equals(product.getProductType())) {
    threshold = 20;
} else {
    threshold = 10;
}

// Step 3: Check recent sales
if (!hasRecentSales(product.getId())) {
    continue;
}

// Step 4: Check if stock is low
if (inv.getQuantity() < threshold) {

    // Step 5: Calculate days until stockout
    int avgSales = getAvgSales(product.getId());
    Integer daysUntilStockout = null;
    if (avgSales > 0) {
        daysUntilStockout = inv.getQuantity() / avgSales;
    }

    // Step 6: Get supplier details
    Supplier supplier = getSupplier(product.getId());
    Map<String, Object> alert = new HashMap<>();
    alert.put("product_id", product.getId());
    alert.put("product_name", product.getName());
    alert.put("sku", product.getSku());
    alert.put("warehouse_id", warehouse.getId());
    alert.put("warehouse_name", warehouse.getName());
```

```
            alert.put("current_stock", inv.getQuantity());
            alert.put("threshold", threshold);
            alert.put("days_until_stockout", daysUntilStockout);

            // Supplier data
            Map<String, Object> supplierData = new HashMap<>();
            if (supplier != null) {
                supplierData.put("id", supplier.getId());
                supplierData.put("name", supplier.getName());
                supplierData.put("contact_email", supplier.getContactEmail());
            }
            alert.put("supplier", supplierData);
            alerts.add(alert);
        }
    }
    Map<String, Object> response = new HashMap<>();
    response.put("alerts", alerts);
    response.put("total_alerts", alerts.size());
    return response;
}
```

Edge Cases Considered:-

- Product has no recent sales → skipped
- Supplier is null → handled safely
- Average sales = 0 → avoid division error
- Inventory is 0 or negative
- Company has no warehouses → empty response
- Large data → can optimize using pagination

Assumptions:-

- Recent sales means last 30 days

- Threshold depends on product type
- Each product has at least one supplier
- Average daily sales is available
- inventoryRepository.findByCompanyId() returns all inventories for that company

Explanation of Logic:-

- First, I fetch all inventory records linked to the company
- Then I check if stock is below the threshold
- I filter only products with recent sales
- I calculate estimated stockout days using average sales
- I attach supplier details for reordering
- Finally, I return the list of alerts with total count.

This implementation helps businesses identify low-stock products efficiently and take timely actions for restocking.